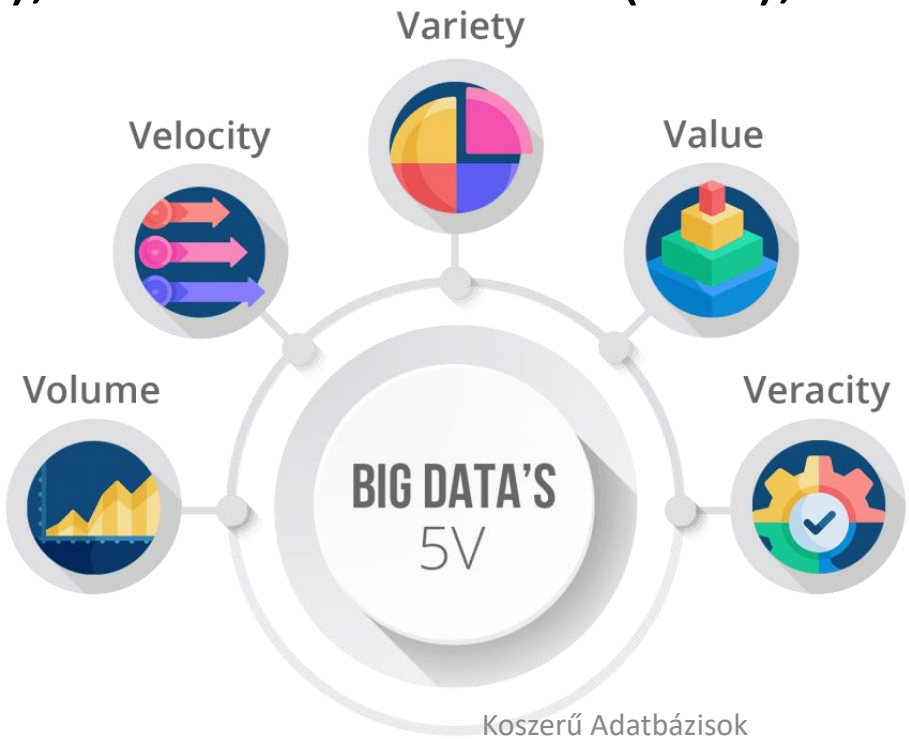
Multi-modell adatbázisok, ArangoDB

Gombos Gergő

based on: Jiaheng Lu and Irena Holubová: Multi-model Data Management

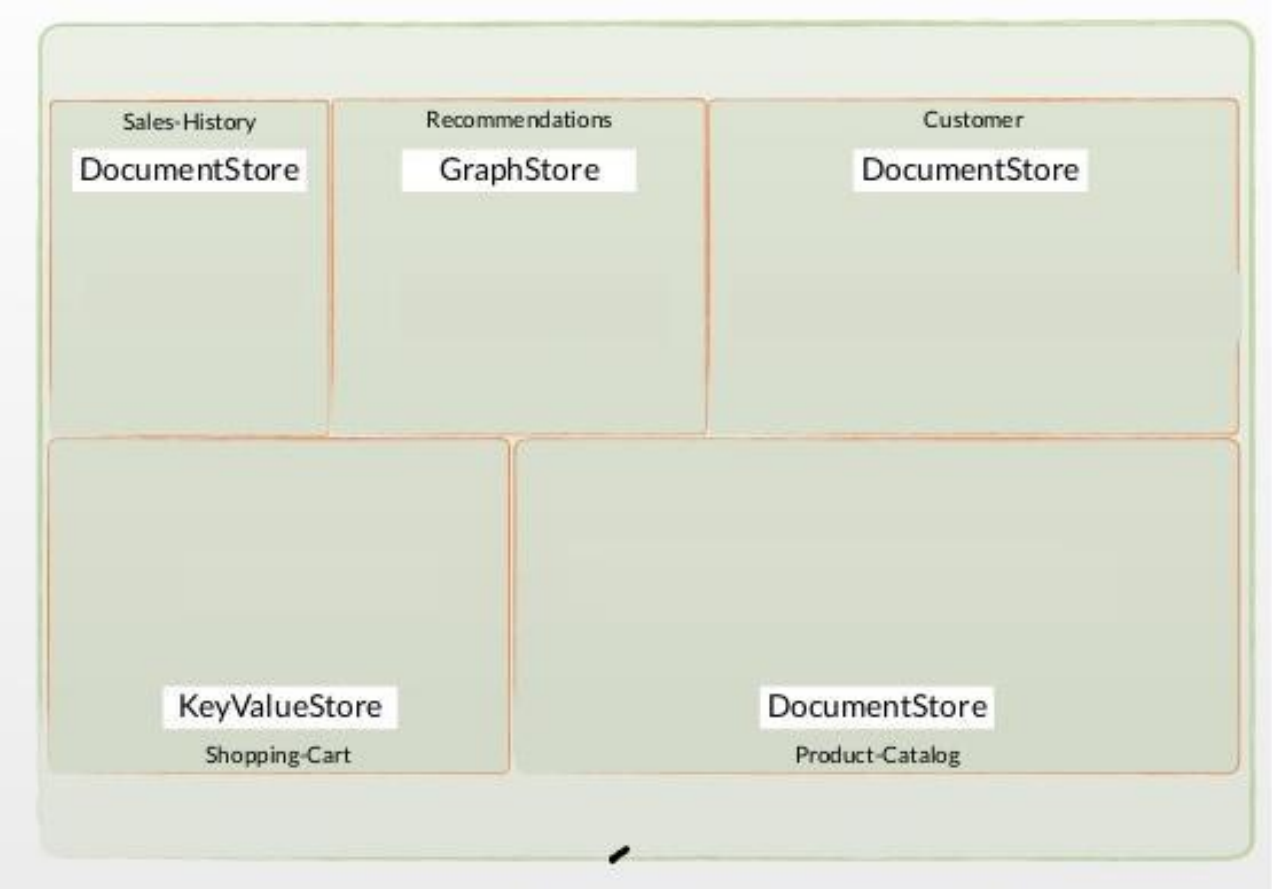
Big Data egyik kihívása: Változatosság (Variety)

- Big Data V4+1 : Volume, Variety, Velocity, Veracity, (Value)
- Változatosság: fa adatok (XML, JSON), gráf adatok (Property graph, RDF, networks), táblázatos adatok (CSV), térbeli adatok, szöveges adatok

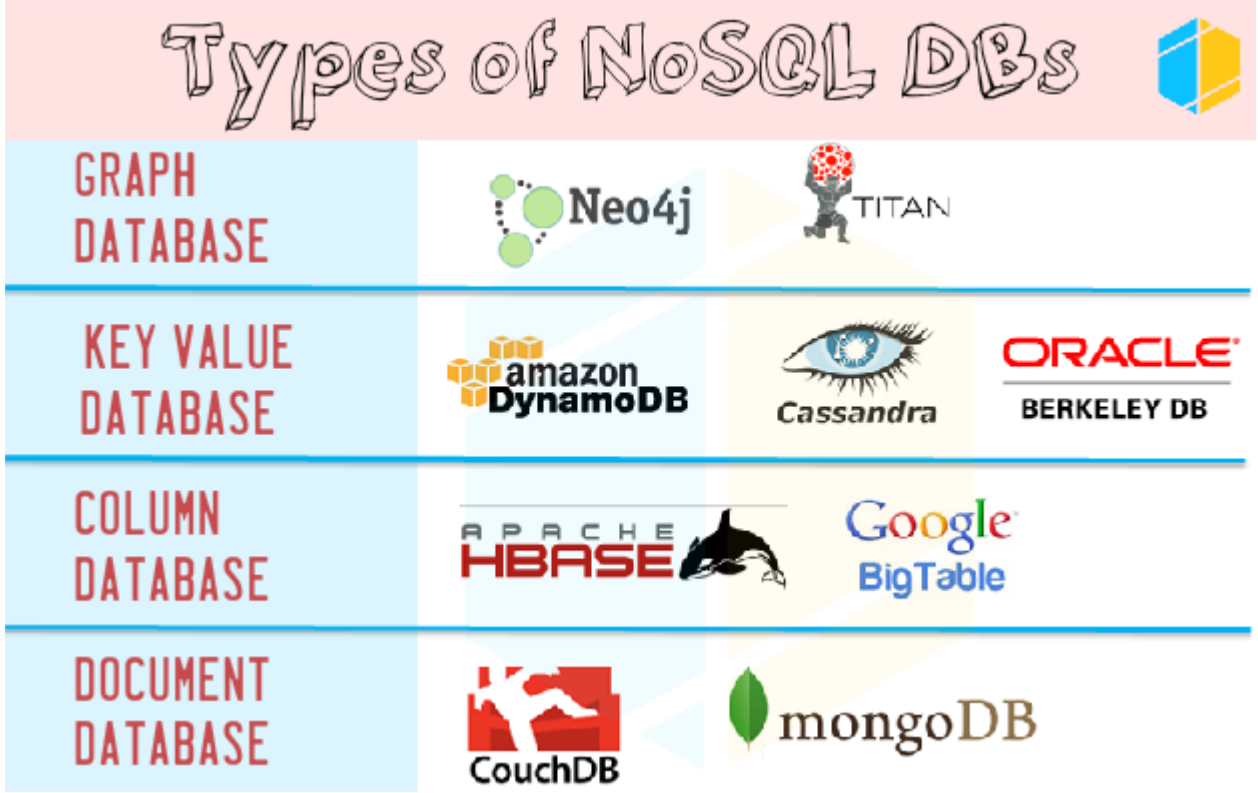


Motiváció: példa alkalmazás

- Webshop példa
  - Sales:
    - dokumentum
  - Ajánlás:
    - gráf
  - Vásárló:
    - dokumentum
  - Kosár:
    - kulcs-érték
  - Termék:
    - dokumentum

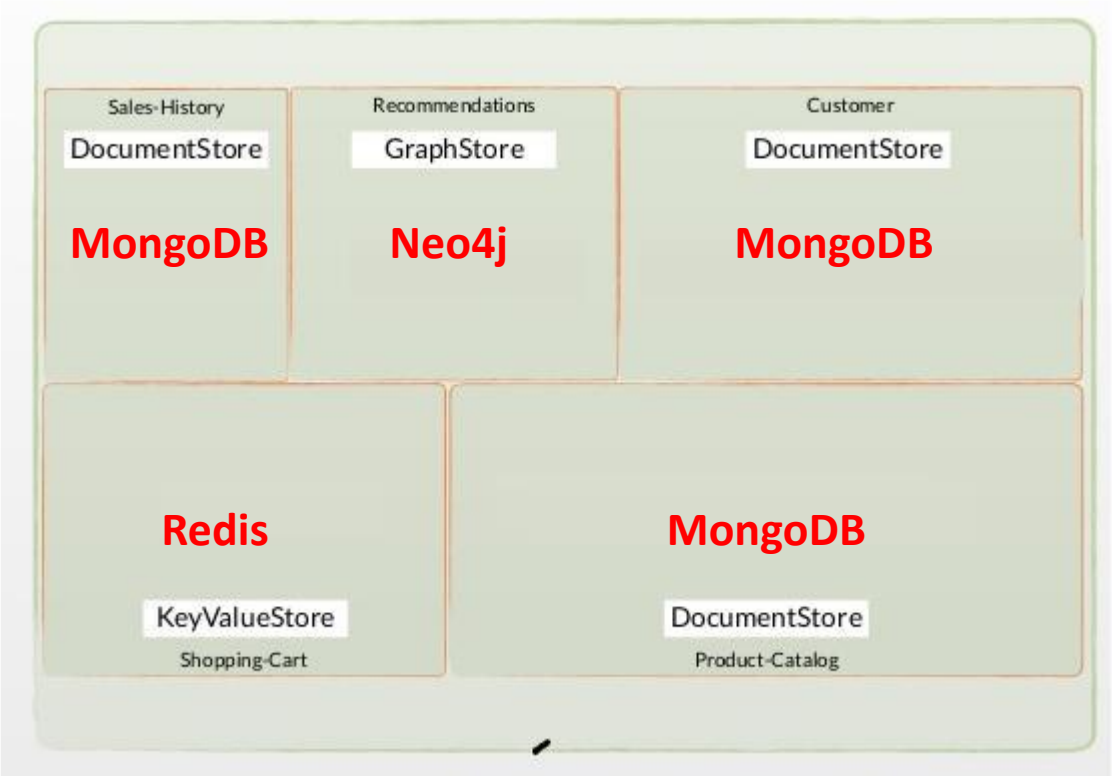


NoSql adattípusok



- Photo downloaded from: http://www.vikramtakkar.com/2015/12/nosql-types-of-nosql-database-part-2.html

Multiple NoSql database



Polyglot Perzisztencia

- A polyglot perzisztencia egy fogalmi kifejezés, amely a különböző adattárolási megközelítések és technológiák alkalmazását jelenti a vállalati alkalmazásokban lévő különféle adattípusok egyedi tárolási követelményeinek támogatására.

- Ha strukturált adatod van:
  - Dokumentum adatbázis

- Ha kapcsolatok vannak az egyedek között és azt akarod hatékonyan lekérdezni:
  - Gráf adatbázis

- Ha magad akarod kezelni az adatok struktúráját és nem akarsz bonyolult lekérdezéseket:
  - Kulcs-érték adatbázis

Előny - Hátrány

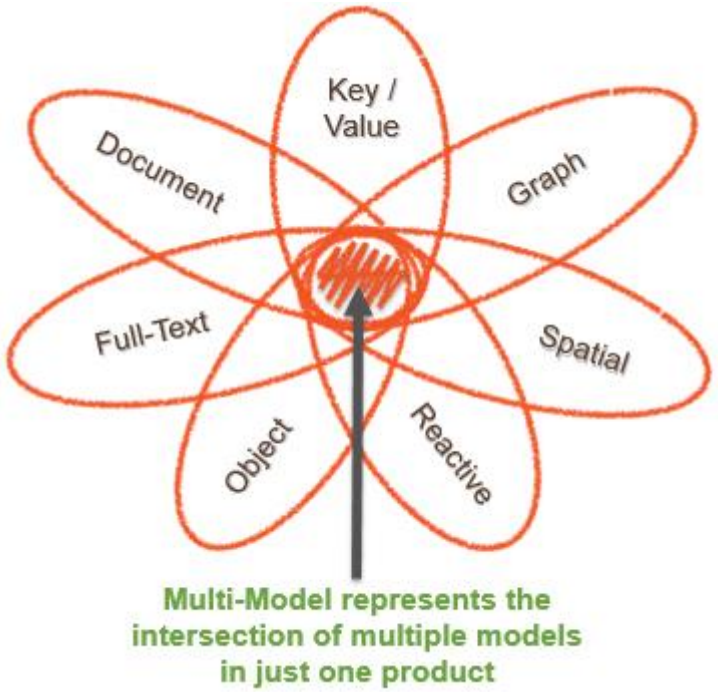
- Több adattípus kezelés
- Jól skálázódik az alkalmazás
- Nagy tapasztalat többféle adatbázis kezelésében

- Több ember kell, hogy a különböző adatbázisokat integrálni tudja
- Fejlesztőknek ismerniük kell a különböző adatbázisokat
- Nehezen kezelhető a több modelleken keresztüli lekérdezések

Multi-modell DB

- A multi-model adatbázis arra lett tervezve, hogy támogatni tudjon többféle modelt egy integrált backend segítségével.

- Dokumentumok, gráfok, kulcs-érték modellek lehetnek amelyeket egy multi-model adatbázisnak kezelnie kell.



Multi-modell vs Multi-modal

- Multi-modell: több modell / típus
  - pl: gráf, fa, relációs, kulcs-érték,…
- Multi-modal: több formátum
  - video, kép, hang, szenzor jelek

Adatbázis motor tulajdonságai

- Egy nem tud megoldani mindent (One size cannot fit all)
- Egy mindent meg tud oldani (One size can fit all)
- Egy sokmindent meg tud oldani (One size fits bunch)

Adatbázis motor tulajdonságai

- One size cannot fit all
  - „SQL analytics, real-time decision support, and data warehouses cannot be supported in one database engine”

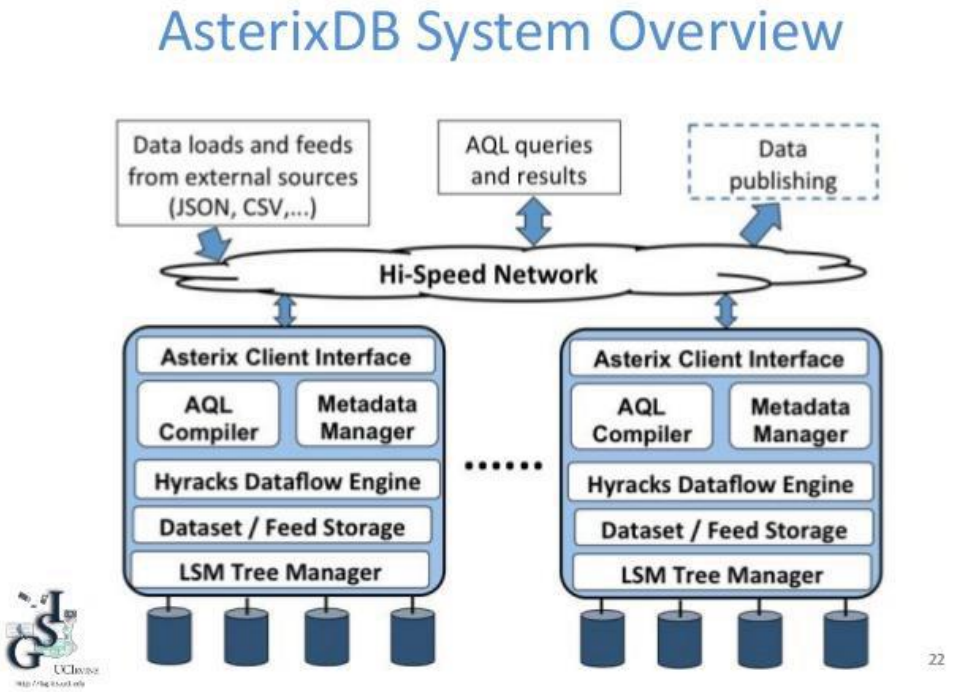
Adatbázis motor tulajdonságai

- One size can fit all - OctopusDB
- „OctopusDB suggests a unified, one size fits all data processing architecture for OLTP, OLAP, streaming systems, and scan-oriented database systems.”

- Minden adat egy központi logban tárolódik
- A logok alapján többféle storage view lehet készíteni
- A lekérdezés optimalizálás, index kiválasztás, stb egy storage view választási probléma

Adatbázis motor tulajdonságai

- One size fits bunch – AsterixDB
  - Párhuzamos féligstruktúrált adatbázis saját adattárolási megoldással, futtatással, lekérdezési nyelvvel, lekérdezés optimalitzációval, JSON és CSV támogatással
  - Adatmodel rugalmas

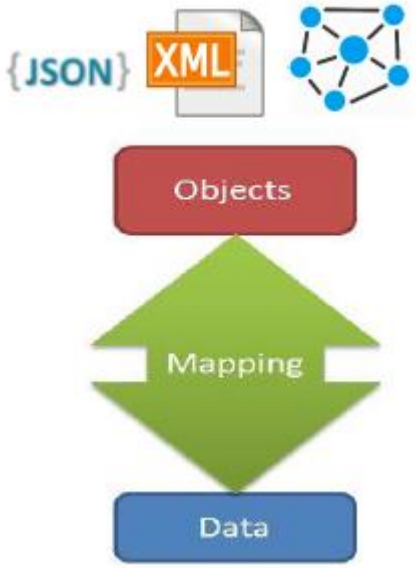


Multi-model adatbázisok

Multi-modell adatbázisok nem újdonságok

- Hasonlóak az objektum-orientált adatbázisokhoz (ORDBMS)

- Az ORDBMS-ben a felhasználók saját tématerületi, alkalmazásspecifikus adatmodellt készíthettek saját függvényekkel, típusokkal és indexekkel.



Előny-hátrány multi-modell adatbázisokra

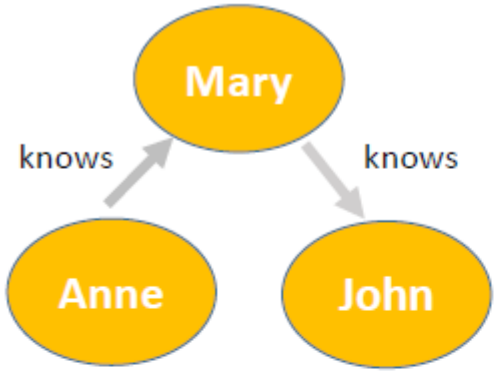
- Több modellt képes kezelni
- Egy rendszer kezeli a hibakezelést
- Egy rendszer kezeli konzisztenciát a modellek között
- Egységes lekérdezőnyelv a multi-modell adatokhoz

- Komplex rendszer
- Kiforratlan még
- Sok kihívás és problémák még

ArangoDB

- ArangoDB egy open source, multi-modell adatbázis több modellel: dokumentum, gráf, kulcs-érték
- Minden adatot dokumentumként tárol
- Mivel a gráf élei és csúcsai is dokumentumok, emiatt megengedi, hogy keverjük a különböző formátumokat (kulcs-érték, JSON, gráf)

Példa multi-model adat és lekérdezés



Social network graph

"1" -- > "34e5e759"

"2"-- > "0c6df508"

Shopping-cart key-value pairs

Customer_ID → Order_no

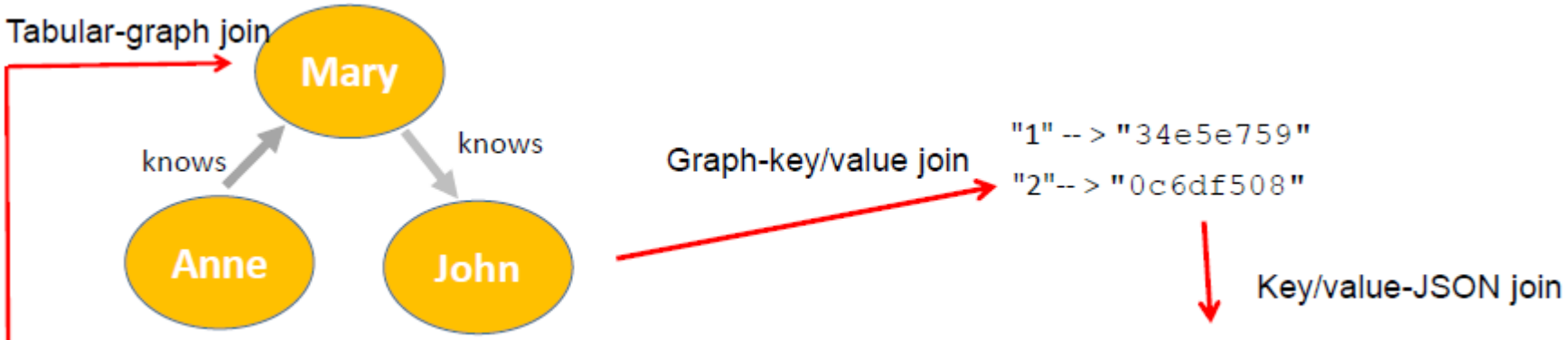
```
{"Order_no":"0c6df508",
 "Orderlines": [
   { "Product_no":"2724f"
     "Product_Name":"Toy",
     "Price":66 },
   { "Product_no":"3424g",
     "Product_Name":"Book",
     "Price":40 } ]
}
```

Order JSON document

Customer relation

Customer_ID	Name	Credit_limit
1	Mary	5,000
2	John	3,000
3	William	2,000

Példa multi-model adat és lekérdezés



Customer_ID	Name	Credit_limit
1	Mary	5,000
2	John	3,000
3	Anne	2,000

```
{"Order_no":"0c6df508",
 "Orderlines": [
   { "Product_no":"2724f"
     "Product_Name":"Toy",
     "Price":66 },
   { "Product_no":"3424g",
     "Product_Name":"Book",
     "Price":40 } ]
}
```

Recommendation query:
Return all product_no which are ordered by a friend of a customer whose credit_limit>3000

Példa multi-model adat és lekérdezés (ArangoDB)

Description: Return all products which are ordered by a friend of a customer whosecredit_limit>3000

```
Let CustomerIDs=(FOR Customer IN Customers FILTER Customer.CreditLimit>
3000 RETURN Customer.id)
Let FriendIDs=(FOR CustomerIDin CustomerIDsFOR Friend IN
1..1OUTBOUNDCustomerIDKnows return Friend.id)
For Friend in FriendIDs
      For Order in 1..1 OUTBOUND Friend Customer2Order
Return Order.orderlines[*].Product_no
```

Result: `["2724f", "3424g"]`

OrientDB

- Gráf, dokumentum, kulcs-érték és objektum modellt támogat
- Kapcsolatokat gráfként kezeli irányított élekkel a rekordok között
- Lekérdezések SQL-szerűek kiegészítve a gráfbejárással.

Példa multi-model adat és lekérdezés (OrientDB)

Description: Return all products which are ordered by a friend of a customer whosecredit_limit>3000

```
Select expand(out("Knows").Orders.orderlines.Product_no) from Customers
where Credit_limit> 3000
```

Result: `["2724f", "3424g"]`

Multi-modell adattárolás – Relációs DB

	Formats	Storage strategy	Query languages	Indices	Scale out	Flexible schema	Comb. data	Cloud
PostgreSQL	relational, key/value, JSON, XML	relational tables - text or binary format + indices	SQL ext.	inverted	N	Y	Y	N
SQL Server	relational, XML, JSON, ...	text, relational tables	SQL ext.	B-tree, full-text	Y	Y	Y	N
IBM DB2	relational, XML, RDF	native XML type / relations for RDF	Extended SQL / XML / SPARQL 1.0/1.1	XML paths / B+ tree, fulltext	Y	Y	Y	N
Oracle DB	relational, XML, JSON	relational, native XML	SQL/XML, JSON SQL ext.	bitmap, B+ tree, function-based, XMLIndex	Y	N	Y	Y
Oracle MySQL	relational, key/value	relational	SQL, memcached API	B-tree	Y	N	Y	Y
Sinew	relational, key/value, nested document, ...	logically a universal relation, physically partially materialized	SQL	-	-	Y	Y	N

Multi-modell adattárolás – Relációs DB

- példa PostgreSQL

```
CREATE TABLE customer(
id INTEGER PRIMARY KEY,
name VARCHAR(50),
address VARCHAR(50),
orders JSONB
);
```

```
INSERT INTO customer
VALUES (1, 'Mary', 'Prague',
'{"Order_no":"0c6df508",
"Orderlines":[
{"Product_no":"2724f", "Product_Name":"Toy","Price":66 },
{"Product_no":"3424g", "Product_Name":"Book", "Price":40}]
}');
INSERT INTO customer
VALUES (2, 'John', 'Helsinki',
'{"Order_no":"0c6df511",
"Orderlines":[
{ "Product_no":"2454f", "Product_Name":"Computer", "Price":34 }]
}');
```

id integer	name character varying (50)	address character varying (50)	orders jsonb
1	Mary	Prague	{"Orderlines":[{"Price":66,"Product_Name":"Toy","Product_no":"2724f"},{"Price":40,"Product_Name":...
2	John	Helsinki	{"Orderlines":[{"Price":34,"Product_Name":"Computer","Product_no":"2454f"}],"Order_no":"0c6df511"}

Multi-modell adattárolás – Relációs DB

- `SELECT json_build_object('id',id,'name',name,'orders',orders) FROM customer;`

json_build_object json
{"orders":{"Orderlines":[{"Price":66,"Product_Name":"Toy","Product_no":"2724f"},{"Price":40,"Product_Name":"Book","Product_no":"3...
{"orders":{"Orderlines":[{"Price":34,"Product_Name":"Computer","Product_no":"2454f"}],"Order_no":"0c6df511"},"id":2,"name":"John"}

- `SELECT jsonb_each(orders) FROM customer;`

jsonb_each record
(Order_no,"""0c6df508""")
(Orderlines,"[{""Price"": 66, ""Product_no"": ""2724f"", ""Product_Name"": ""To...
(Order_no,"""0c6df511""")
(Orderlines,"[{""Price"": 34, ""Product_no"": ""2454f"", ""Product_Name"": ""Co...

- `SELECT jsonb_object_keys(orders) FROM customer;`

jsonb_object_keys text
Order_no
Orderlines
Order_no
Orderlines

Multi-modell adattárolás – Oszlop DB

	Formats	Storage strategy	Query languages	Indices	Scale out	Flexible schema	Comb. data	Cloud
Cassandra	text, user-defined type	sparse tables	SQL-like CQL	inverted, B+ tree	Y	N	Y	Y
CrateDB	relational, JSON, BLOB, arrays	columnar store based on Lucene and Elasticsearch	SQL	Lucene	Y	Y	Y	N
DynamoDB	key/value, document (JSON)	column store	simple API (get / put / update) + simple queries over indices	hashing	Y	Y	Y	Y
HPE Vertica	JSON, CSV	flex tables + map	SQL-like	for materialized data	Y	Y	Y	N

Multi-modell adattárolás – Oszlop DB

- példa Cassandra

```
create keyspace myspace;

CREATE TYPE myspace.orderline(
  product_notext,
  product_nametext,
  pricefloat
);

CREATE TYPE myspace.myorder(
  order_notext,
  orderlineslist<frozen<orderline>>
);

CREATE TABLE myspace.customer(
  id INT PRIMARY KEY,
  nametext,
  addresstext,
  orderslist<frozen<myorder>>
);
```

```
INSERT INTO myspace.customer JSON
' {"id":1,
"name":"Mary",
"address":"Prague",
"orders" : [
  { "order_no":"0c6df508",
      "orderlines":[
        { "product_no" : "2724f",
          "product_name" : "Toy",
          "price" : 66 },
        { "product_no" : "3424g",
          "product_name" :"Book",
          "price" : 40 } ] } ]
}';
```

```
INSERT INTO myspace.customer JSON
' {"id":2,
"name":"John",
"address":"Helsinki",
"orders" : [
  { "order_no":"0c6df511",
    "orderlines":[
    { "product_no" : "2454f",
      "product_name" : "Computer",
      "price" : 34 } ] } ]
}';
```

Multi-modell adattárolás – Oszlop DB

- példa Cassandra

```
CREATE TABLE myspace.users (
  id text PRIMARY KEY,
  age int,
  countrytext
);

INSERT INTO myspace.users (id, age, state) VALUES ('Irena', 37, 'CZ’);

SELECT JSON * FROM myspace.users;


[json]
-------------------------------------------
{"id": "Irena", "age": 37, "country": "CZ"}
```

Multi-modell adattárolás – kulcs- érték DB

	Formats	Storage strategy	Query languages	Indices	Scale out	Flexible schema	Comb. data	Cloud
Riak	key/value, XML, JSON	key/value pairs in buckets	Solr	Solr	Y	N	Y	N
c-treeACE	key/value + SQL API	record-oriented ISAM	SQL	ISAM	Y	Y	-	N
Oracle NoSQL DB	key/value, (hierarchical) table API	key/value	SQL	B-tree	Y	N	Y	N

Multi-modell adattárolás – kulcs- érték DB

- példa Oracle NoSQL

```
create table Customers (
 id integer,
 name string,
 address string,
 orders array (
   record (
     order_no string,
     orderlines array (
        record (
          product_no string,
          product_name string,
          price integer )
     )
)),primary key (id));

import -table Customers -file customer.json
```

```
{ "id":1,
  "name":"Mary",
  "address":"Prague",
  "orders" : [
    { "order_no":"0c6df508",
      "orderlines":[
        { "product_no" : "2724f",
          "product_name" : "Toy",
          "price" : 66 },
        { "product_no" : "3424g",
          "product_name":"Book",
          "price" : 40 } ] } ]}
{ "id":2,
  "name":"John",
  "address":"Helsinki",
  "orders" : [
    { "order_no":"0c6df511",
      "orderlines":[
        { "product_no" : "2454f",
          "product_name" : "Computer",
          "price" : 34} ] } ] }
```

Multi-modell adattárolás – kulcs- érték DB

- példa Oracle NoSQL

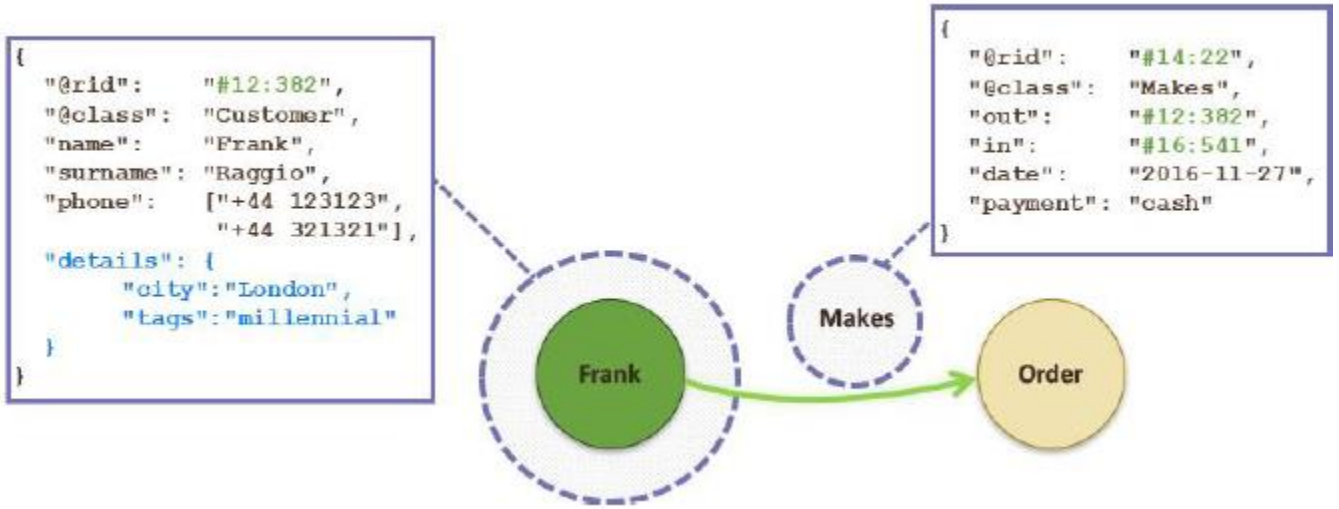
```
sql-> select * from Customers
  -> ;
 +----+------+----------+-----------------------------+
 | id | name | address  |           orders            |
 +----+------+----------+-----------------------------+
 |  2 | John | Helsinki | order_no         | 0c6df511 |
 |    |      |          | orderlines                  |
 |    |      |          |     product_no   | 2454f    |
 |    |      |          |     product_name | Computer |
 |    |      |          |     price        | 34       |
 +----+------+----------+-----------------------------+
 |  1 | Mary | Prague   | order_no         | 0c6df508 |
 |    |      |          | orderlines                  |
 |    |      |          |     product_no   | 2724f    |
 |    |      |          |     product_name | Toy      |
 |    |      |          |     price        | 66       |
 |    |      |          |                             |
 |    |      |          |     product_no   | 3424g    |
 |    |      |          |     product_name | Book     |
 |    |      |          |     price        | 40       |
 +----+------+----------+-----------------------------+
```

Multi-modell adattárolás – dokumentum DB

	Formats	Storage strategy	Query languages	Indices	Scale out	Flexible schema	Comb. data	Cloud
ArangoDB	key/value, document, graph	document store allowing references	SQL-like AQL	mainly hash (eventually unique or sparse)	Y	Y	Y	N
Couchbase	key/value, document, distributed cache	document store + append-only write	SQL-based N1QL	B+tree, B+trie	Y	Y	Y	N
MarkLogic	XML, JSON, RDF, binary, text, ...	storing like hierarchical XML data	XPath, XQuery, SQL-like	inverted + native XML	Y	Y	Y	N

Multi-modell adattárolás – gráf DB

	Formats	Storage strategy	Query languages	Indices	Scale out	Flexible schema	Comb. data	Cloud
OrientDB	graph, document, key/value, object	key/value pairs + object-oriented links	Gremlin, SQL ext.	SB-tree, ext. hashing, Lucene	Y	Y	Y	N



Multi-modell adattárolás – gráf DB

- példa OrientDB

```
CREATE CLASS orderline EXTENDS V
CREATE PROPERTY orderline.product_no STRING
CREATE PROPERTY orderline.product_name STRING
CREATE PROPERTY orderline.price FLOAT
CREATE CLASS order EXTENDS V
CREATE PROPERTY order.order_no STRING
CREATE PROPERTY order.orderlines EMBEDDEDLIST orderline
CREATE CLASS customer EXTENDS V
CREATE PROPERTY customer.id INTEGER
CREATE PROPERTY customer.name STRING
CREATE PROPERTY customer.address STRING
CREATE CLASS orders EXTENDS E
CREATE CLASS knows EXTENDS E
```

Multi-modell adattárolás – gráf DB

- példa OrientDB

```
CREATE VERTEX order CONTENT {
   "order_no":"0c6df508",
   "orderlines":[
    { "@type":"d",
      "@class":"orderline",
      "product_no":"2724f",
      "product_name":"Toy",
      "price":66 },
    { "@type":"d",
      "@class":"orderline",
      "product_no":"3424g",
      "product_name":"Book",
      "price":40}]
}
```

```
CREATE VERTEX order CONTENT {
   "order_no":"0c6df511",
   "orderlines":[
    { "@type":"d",
      "@class":"orderline",
      "product_no":"2454f",
      "product_name":"Computer",
      "price":34 }]
}
```

```
CREATE VERTEX customer CONTENT {
  "id" : 1,
  "name" : "Mary",
  "address" : "Prague"
  }

CREATE VERTEX customer CONTENT {
  "id" : 2,
  "name" : "John",
  "address" : "Helsinki"
  }
```

Multi-modell adattárolás – gráf DB

- példa OrientDB

```
CREATE EDGE orders FROM
   (SELECT FROM customer WHERE name = "Mary")
   TO
   (SELECT FROM order WHERE order_no = "0c6df508")

CREATE EDGE orders FROM
   (SELECT FROM customer WHERE name = "John")
   TO
   (SELECT FROM order WHERE order_no = "0c6df511")

CREATE EDGE knows FROM
   (SELECT FROM customer WHERE name = "Mary")
   TO
   (SELECT FROM customer WHERE name = "John")
```

Multi-modell lekérdezőnyelvek

PostgreSQL	relational	Getting an array element by index, an object field by key, an object at a specified path, containment of values/paths, top-level key-existence, deleting a key/value pair / a string element / an array element with specified index /a field / an element with specified path,...
SQL Server	relational	JSON: export relational data in the JSON format, test JSON format of a text value, JavaScript-like path queries, SQLXML: SQL view of XML data + XML view of SQL relations
IBM DB2	relational	SQL/XML + e.g. embedding SQL queries to XQuery expressions
Oracle DB	relational	SQL/XML + JSON extensions (`JSON_VALUE`, `JSON_QUERY`, `JSON_EXISTS`,...)
Couchbase	document	Classical clauses such as `SELECT`, `FROM` (multiple buckets), ... for JSON
ArangoDB	document	key/value: insert, look-up, update document: simple QBE, complex joins, functions, ... graph: traversals, shortest path searches
Oracle NoSQL DB	key/value	SQL-like, extended for nested data structures
c-treeACE	key/value	SQL-like language
Cassandra	column	`SELECT`, `FROM`, `WHERE`, `ORDER BY`, `LIMIT` with limitations
CrateDB	column	Standard ANSI SQL 92 + usage nested JSON attributes
OrientDB	graph	Classical joins not supported, the links are simply navigated using dot notation; main SQL clauses + nested queries
Caché	object	SQL + object extensions (e.g. object references instead of joins)

Multi-modell lekérdezőnyelvek

- Oracle

```
sql-> SELECT c.name, c.orders.order_no, c.orders.orderlines[0].product_name
   -> FROM customers c
   -> where c.orders.orderlines[0].price > 50;
 +------+----------+--------------+
 | name | order_no | product_name |
 +------+----------+--------------+
 | Mary | 0c6df508 | Toy          |
 +------+----------+--------------+

sql-> SELECT c.name, c.orders.order_no,
   -> [c.orders.orderlines[$element.price >35]]
   -> FROM customers c;
 +------+----------+-------------------------+
 | name | order_no |        Column_3         |
 +------+----------+-------------------------+
 | Mary | 0c6df508 | product_no    | 2724f   |
 |      |          | product_name  | Toy     |
 |      |          | price         | 66      |
 |      |          |                         |
 |      |          | product_no    | 3424g   |
 |      |          | product_name  | Book    |
 |      |          | price         | 40      |
 +------+----------+-------------------------+
 | John | 0c6df511 |                         |
 +------+----------+-------------------------+
```

```
sql-> select * from Customers
   -> ;
 +----+------+----------+----------------------------+
 | id | name | address  |           orders           |
 +----+------+----------+----------------------------+
 |  2 | John | Helsinki | order_no         | 0c6df511 |
 |    |      |          | orderlines                  |
 |    |      |          |     product_no   | 2454f    |
 |    |      |          |     product_name | Computer |
 |    |      |          |     price        | 34       |
 +----+------+----------+----------------------------+
 |  1 | Mary | Prague   | order_no         | 0c6df508 |
 |    |      |          | orderlines                  |
 |    |      |          |     product_no   | 2724f    |
 |    |      |          |     product_name | Toy      |
 |    |      |          |     price        | 66       |
 |    |      |          |                             |
 |    |      |          |     product_no   | 3424g    |
 |    |      |          |     product_name | Book     |
 |    |      |          |     price        | 40       |
 +----+------+----------+----------------------------+
```

Multi-modell lekérdezőnyelvek

- PostgreSQL

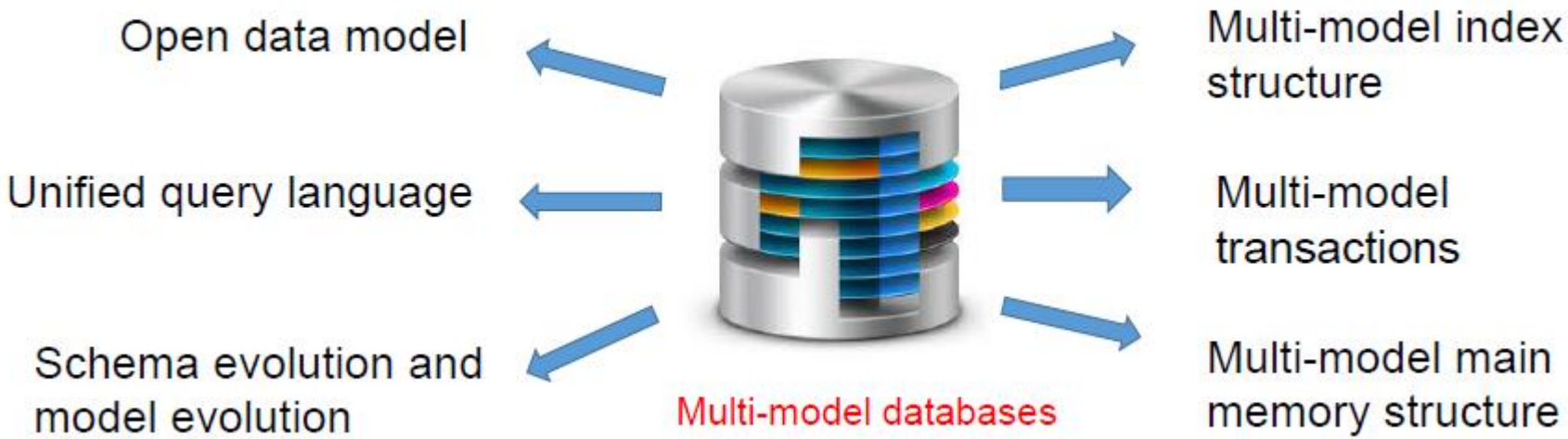
id integer	name character varying (50)	address character varying (50)	orders jsonb
1	Mary	Prague	{"Orderlines":[{"Price":66,"Product_Name":"Toy","Product_no":"2724f"},{"Price":40,"Product_Name":...
2	John	Helsinki	{"Orderlines":[{"Price":34,"Product_Name":"Computer","Product_no":"2454f"}],"Order_no":"0c6df511"}

```
{"Order_no":"0c6df508",
   "Orderlines":[
    { "Product_no":"2724f",
      "Product_Name":"Toy",
      "Price":66 },
    {"Product_no":"3424g",
     "Product_Name":"Book",
     "Price":40}]
}
```

```
SELECT name,
orders->>'Order_no' as Order_no,
orders#>'{Orderlines,1}'->>'Product_Name' as Product_Name
FROM customer
where orders->>'Order_no' <> '0c6df511';
```

name character varying (50)	order_no text	product_name text
Mary	0c6df508	Book

Multi-modell kihívások

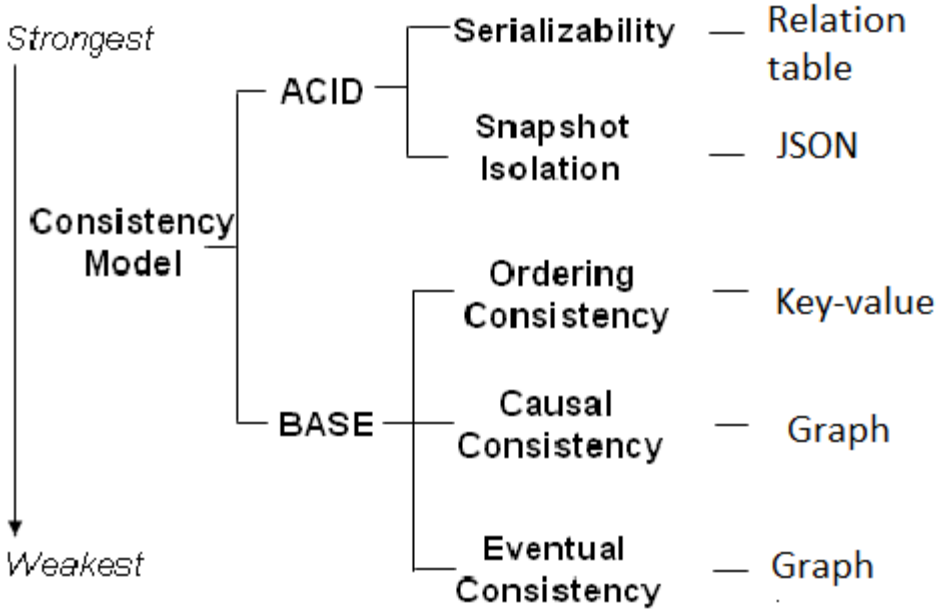


Multi-modell kihívások

- Nyitott adatmodell:
  - Rugalmas adatmodell kell, amely tudja kezelni, a különböző forrásokból származó adatokat.
- Egységes lekérdezőnyelv:
  - Olyan nyelv kell, amely minden modellt le tud kérdezni egyszerre
- Modell evolúció
  - A modellek változhatnak, ehhez szükséges a megfeleltetés a régi és új modell között
- Multi-modell indexelés
  - Hogyan indexeljünk több modell között? Olyan index kell, amely össze tudja kötni a gráf, dokumentum vagy épp a relációs adatokat is.

Multi-modell kihívások

- Multi-modell memória kezelés
  - Mit és hogyan tároljunk a memóriában?
- Multi-modell tranzakciókezelés
  - A különböző NoSQL adatbázisok különböző konzisztencia szintet határoznak meg.
  - Hogyan kezeljük a tranzakciókat?



ArangoDB

ArangoDB

- ArangoDB JSON objektumként tárolja a gráfokat és a dokumentumokat amelyek kollekciókba vannak szervezbe az adatbázisban
- A dokumentumok (adat rekordok) kollekciókban vannak, a kollekciók adatbázisban
- Minden rekord egy JSON objektum
- A dokumentumok bináris formátumban vannak tárolva a háttérben: VelocyPack (gyors és kompakt szerializált formátum).

```
{
  "name": "ArangoDB",
  "tags": ["graph", "database", "NoSQL"],
  "scalable": true,
  "company": {
    "name": "ArangoDB Inc.",
    "founded": 2015
  }
}
```

ArangoDB

- Data Models
  - Key-Value Modell
    - Dokument adatmodell része
    - Dokumentum kulcs (document key)
      - Minden dokumentum rendelkezik egy _key attribútommal a kollekciókon belül
      - Ez egy primary key
      - String

    - Dokumentum azonosító (document identifier)
      - Egyedi azonosítója (_id) a dokumentumnak az adatbázison belül.
      - Ez egy kombinációja a kollekció nevének és a dokumentum kulcsnak
      - Ezt használja primary key-nek a háttérben

ArangoDB

- Data Models
  - Document modell
    - JSON objektumként tárolja az adatrekordokat
    - Lehet keresni bármely attribútumra indexszel és anélkül is
  - Graph modell
    - Élek és a csúcsok is dokumentumként vannak tárolva.
    - Az éleknek két speciális attribútuma van: _from, _to
    - A csúcsok és az élek is akárhány attributummal rendelkezhetnek. - Labeled Property Graph
    - Az élek mindig irányítottak
    - Lehetnek párhuzamos élek
    - Rendelkezik gráf algoritmusokkal: legrövidebb út keresés
    - Támogatja az elosztott gráf algoritmust is: Pregel

ArangoDB -AQL

- Create

```
INSERT {
    "name": "Ned",
    "surname": "Stark",
    "alive": true,
    "age": 41,
    "traits": ["A","H
} INTO Characters
```

```
FOR c IN Characters
    RETURN c
```

- Read

```
FOR c IN Characters
    RETURN c.name
```

```
FOR c IN Characters
    FILTER c.name == "Ned"
    RETURN c._key
```

- Update

```
UPDATE @key
WITH { alive: false}
IN Characters
```

```
REPLACE @key WITH {
    name: "Ned",
    surname: "Stark",
    alive: false,
    age: 41,
    traits: ["A","H","C","N","P"]
} IN Characters
```

```
FOR c IN Characters
    UPDATE c WITH { season: 1 } IN Characters
"""
```

- Delete

```
REMOVE @key IN Characters
"""
```

```
FOR c IN Characters
    REMOVE c IN Characters
```

ArangoDB- AQL

- SORT

```
FOR c IN Characters
    SORT c.name
    RETURN c.name
```

```
FOR c IN Characters
    SORT c.name DESC
    RETURN c.name
```

```
FOR c IN Characters
    FILTER c.surname
    SORT c.surname, c.name
    LIMIT 10
    RETURN {
        surname: c.surname,
        name: c.name
    }
```

- SKIP, LIMIT

```
FOR c IN Characters
    SORT c.name
    LIMIT 5
    RETURN c.name
```

```
FOR c IN Characters
    SORT c.name
    LIMIT 2, 5
    RETURN c.name
```

- JOIN

```
FOR c IN Characters
    LIMIT 5
    RETURN DOCUMENT("Traits", c.traits)
```

```
FOR c IN Characters
    LIMIT 5
    RETURN DOCUMENT("Traits", c.traits)[*].en
```

```
FOR c IN Characters
  LIMIT 5
  RETURN MERGE(c, {
    traits: (
      FOR key IN c.traits
        FOR t IN Traits
          FILTER t._key == key
          RETURN t.en
    )
  })
```

ArangoDB- AQL

- Subqueries

```
FOR p IN persons
  LET recommendations = ( // subquery start
    FOR r IN recommendations
      FILTER p.id == r.personId
      SORT p.rank DESC
      LIMIT 10
      RETURN r
  ) // subquery end
  RETURN { person : p, recommendations : recommendations }
```

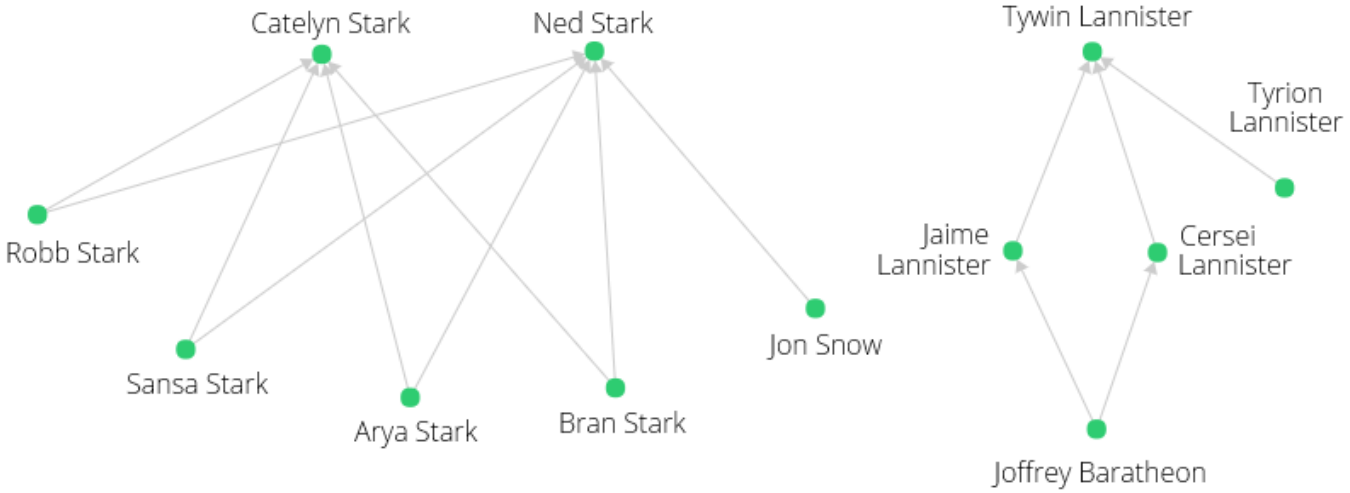
```
FOR p IN persons
  COLLECT city = p.city INTO g
  RETURN {
    city : city,
    numPersons : LENGTH(g),
    maxRating: MAX( // subquery start
      FOR r IN g
      RETURN r.p.rating
    ) // subquery end
  }
```

ArangoDB- AQL

- Gráf készítés

```
    }, {
        "parent": { "name": "Jaime", "surname": "Lannister" },
        "child": { "name": "Joffrey", "surname": "Baratheon" }
    }
]

FOR rel in data
    LET parentId = FIRST(
        FOR c IN Characters
            FILTER c.name == rel.parent.name
            FILTER c.surname == rel.parent.surname
            LIMIT 1
            RETURN c._id
    )
    LET childId = FIRST(
        FOR c IN Characters
            FILTER c.name == rel.child.name
            FILTER c.surname == rel.child.surname
            LIMIT 1
            RETURN c._id
    )
    FILTER parentId != null AND childId != null
    INSERT { _from: childId, _to: parentId } INTO ChildOf
    RETURN NEW
```



ArangoDB- AQL

- Gráf lekérdezések

```
sansa_parents_query = """
// First find the start node, i.e., sansa
FOR c IN Characters
    FILTER c.name == "Sansa"
    // Then start a Graph traversal from that start node
    FOR v IN 1..1 OUTBOUND c ChildOf
    RETURN v.name
```

```
ned_children_query = """
// First find the start node, i.e., ned
FOR c IN Characters
    FILTER c.name == "Ned"
    // Then start a Graph traversal from that start node
    FOR v IN 1..1 INBOUND c ChildOf
    RETURN v.name
```

```
tywin_grandchildren_query = """
// First find the start node, i.e., ned
FOR c IN Characters
    FILTER c.name == "Tywin"
    // Then start a Graph traversal from that start node
    FOR v IN 2..2 INBOUND c ChildOf
    RETURN v.name
```

```
joffrey_ancestors_query = """
FOR c IN Characters
    FILTER c.name == "Joffrey"
    FOR v IN 1..2 OUTBOUND c ChildOf
        RETURN DISTINCT v.name
```

ArangoDB- AQL

```
LET places = [
    { "name": "Dragonstone", "coordinate": [ 55.167801, -6.815096 ] },
    { "name": "King's Landing", "coordinate": [ 42.639752, 18.110189 ] },
    { "name": "The Red Keep", "coordinate": [ 35.896447, 14.446442 ] },
    { "name": "Yunkai", "coordinate": [ 31.046642, -7.129532 ] },
    { "name": "Astapor", "coordinate": [ 31.50974, -9.774249 ] },
    { "name": "Winterfell", "coordinate": [ 54.368321, -5.581312 ] },
    { "name": "Vaes Dothrak", "coordinate": [ 54.16776, -6.096125 ] },
    { "name": "Beyond the wall", "coordinate": [ 64.265473, -21.094093 ] }
]


FOR place IN places
    INSERT place INTO Locations
```

- Geospatial lekérdezések
  - 3 legközelebbi pontot

```
FOR loc IN NEAR(Locations, 53.35, -6.26, 3)
    RETURN {
        name: loc.name,
        latitude: loc.coordinate[0],
        longitude: loc.coordinate[1]
    }
```

  - 200 kmen belüli pontok

```
FOR loc IN WITHIN(Locations, 53.35, -6.26, 200 * 1000)
    RETURN {
        name: loc.name,
        latitude: loc.coordinate[0],
        longitude: loc.coordinate[1]
    }
```

Index készítése

Location.ensureIndex({ type: "geo",
fields: ["geometry"], geoJson:true })

Távolság lekérdezése méterben

```
FOR loc IN NEAR(Locations, 53.35, -6.26, 3, "distance")
    RETURN {
        name: loc.name,
        latitude: loc.coordinate[0],
        longitude: loc.coordinate[1],
        distance: loc.distance / 1000
    }
```

ArangoDB - AQL

- AQL tutorial

- https://www.arangodb.com/docs/stable/aql/index.html

Köszönöm a Figyelmet!